NyayaSetu: Project Overview

An AI-powered petition portal built with React, Flask, SQLite, and a machine learning triage engine to automate civic grievance routing.

Problem Statement

Classification Gap

No automated system exists to classify petition urgency, leading to manual triage and potential delays in identifying critical issues.

Tracking Absence

Citizens have no way to track whether their petition reached the right government department or what action was taken.

Project Objectives

01

AI Petition Classifier

Build an artificial intelligence system to triage petition urgency automatically using machine learning algorithms and natural language processing.

02

Real-Time Tracker

Provide a real-time application tracker for citizens to monitor petition status, department routing, and resolution progress.

Proposed Solution

1

Digitise Petition Filing

Online form replaces paper-based submission

2

ML-Based Urgency Classification

Auto-classify urgency on submission

3

Intelligent Routing

Route petitions to correct department

4

Stage Notification

Notify citizens at every stage

System Architecture

React Frontend

User interface for petition submission and tracking

Flask REST API

Backend service handling requests and responses

SQLite Storage

Persistent storage for all petition records and metadata

ML Pipeline

Machine learning pipeline classifies urgency on submission

Apache Airflow

Schedules daily department dispatch jobs automatically

Technology Stack

Frontend

- React
- Vite

Backend

- Flask
- Flask-CORS

Database

- SQLite

Machine Learning

- SentenceTransformer (all-mpnet-base-v2)
- SVC Classifier
- LabelEncoder

Automation

- Apache Airflow
- SMTP

Petition Filing: 3-Step Form

Personal Information

Collect name, email address, and phone number with input validation to ensure data quality and contactability.

Address Details

Gather street address, city, state, and pincode to enable geographic routing and location-based department assignment.

Petition Content

Enter petition title, category, and detailed description with validation to ensure completeness before submission.

AI Urgency Classification Pipeline

01

Text Normalisation

Input description converted to lowercase for consistent processing

02

Language Detection

langdetect library identifies input language automatically

03

Translation to English

Non-English text translated via GoogleTranslate API

04

Sentence Embedding

SentenceTransformer (all-mpnet-base-v2) generates vector representation

05

Urgency Classification

Trained SVC classifier assigns: Urgent Action Required / Fast Action / Normal Action

Multilingual Support

Automatic Detection

langdetect library identifies the input language from the petition description field.

Intelligent Translation

Non-English text is split into sentences and translated to English before embedding.

Language Coverage

Supports Tamil, Hindi, and additional languages through automated translation pipeline.

Database Design: SQLite

Primary Key

NS-YYYYMMDD-XXXXXXXX format generates unique petition ID

Personal Fields

Name, email, phone, address, city, state, pincode

Petition Fields

Title, category, description, detected language

ML Fields

Predicted label, label ID

Status Fields

Submitted timestamp, dispatched flag, dispatched timestamp

Application Tracker Interface

Citizen Access

Citizens enter their unique petition ID to retrieve real-time status updates

Available Information

- Urgency classification label
- Assigned category
- Location details
- Submission timestamp
- Dispatch status and timestamp

Feature 1: Confirmation Email

Upon successful petition submission, the system automatically sends a confirmation email via SMTP to the petitioner.

01

Unique Petition ID

Generated identifier for tracking purposes

02

Urgency Level

Predicted classification based on content analysis

03

Next Steps

Expected timeline and follow-up instructions

Feature 2: Apache Airflow Daily Workflow

A directed acyclic graph (DAG) executes every 24 hours to process pending petitions and route them to appropriate departments.

Extract Pending

Query SQLite for unnotified records

Classify Labels

Group by urgency classification

Send Emails

Notify mapped departments

Mark Dispatched

Update database status and timestamp

Department Routing Logic

Urgent Action Required

Routed to Emergency Response Department

Response Time: Within 24 hours

Fast Action

Assigned to Operations Team

Response Time: Within 48 hours

Normal Action

Forwarded to Standard Grievance Cell

Response Time: Within 7 days

Key Benefits

Reduced Routing Time

Processing time decreased from multiple days to hours through automated classification and routing

Zero Missed Petitions

Daily Apache Airflow jobs ensure no petition remains unprocessed or forgotten

Citizen Transparency

Real-time tracking via the application tracker provides complete visibility into petition status

Scalable Architecture

System can accommodate any number of departments and categories without architectural changes

System Advantages

Operational Efficiency

- Automated classification eliminates manual sorting
- Daily batch processing reduces administrative burden
- Guaranteed delivery to correct departments

Citizen Experience

- Immediate confirmation with unique ID
- Clear expectations for response timelines
- Self-service tracking reduces inquiry calls

Conclusion

This petition system demonstrates how automated workflows, proper database design, and scheduled batch processing can transform manual grievance handling into an efficient, transparent, and scalable operation. The combination of SQLite for data persistence, SMTP for notifications, and Apache Airflow for orchestration creates a reliable foundation for citizen engagement.